REPORTE TÉCNICO: ENTREGA 1

Local · Red Social de Proximidad Criptográfica

Materia: Desarrollo Móvil (Android)
Aplicación: Local (RedSocialNFC)
Rama GitHub: main (alaanvirtual)
Compilación: BUILD SUCCESSFUL (0 errores)

1. BIBLIOTECAS Y FRAMEWORKS UTILIZADOS

La aplicación **Local** está construida siguiendo la arquitectura recomendada por Google para Android moderno, utilizando tecnologías declarativas y persistencia local sin conexión:

Biblioteca / Framework	Versión	Propósito y Justificación Técnica
Jetpack Compose (BOM)	`2026.02.01`	Framework declarativo moderno de interfaz gráfica para Android. Elimina el uso de XML y permite construir interfaces reactivas basadas en funciones `@Composable` con re-composición inteligente.
Material Design 3 (Material3)	`1.3.x`	Sistema de diseño oficial adaptado a una paleta sobria de morados y grises mate (no luminosos). Provee componentes accesibles como `OutlinedTextField`, `Card`, `Surface` y `Scaffold`.
AndroidX Room Database	`2.8.4`	Capa de abstracción sobre SQLite local. Provee mapeo objeto-relacional (ORM), verificación de consultas SQL en tiempo de compilación y persistencia segura de usuarios sin conexión a internet.
KSP (Kotlin Symbol Processing)	`2.2.10-2.0.2`	Herramienta de procesamiento de anotaciones de alto rendimiento para Kotlin. Genera en tiempo de compilación las implementaciones concretas de `@Dao` y `@Database` de Room.
Kotlin Coroutines & Flow	`1.8.x`	Manejo de programación asíncrona no bloqueante. Ejecuta operaciones pesadas de base de datos en hilos secundarios mediante `suspend fun` y `rememberCoroutineScope()`.
SharedPreferences	Nativo Android	Almacenamiento clave-valor ligero en memoria para persistir la sesión activa y fijar el identificador UUID único del hardware entre reinicios de la aplicación.
Material Icons Extended	`1.7.x`	Catálogo ampliado de iconografía vectorial oficial (candados, ojos de visibilidad, antena NFC, avatares, puertas de salida).
Android SDK & Gradle Plugin	`AGP 9.3.2 / SDK 37`	Herramientas de empaquetado, dexing y compilación del APK nativo con soporte retrocompatible desde Android 7.0 (MinSdk 24).

2. FUNCIONAMIENTO DETALLADO DEL CÓDIGO

La aplicación **Local** implementa una arquitectura desacoplada y limpia acorde a los módulos vistos en el curso:

A. Punto de Entrada y Navegación Reactiva (`MainActivity.kt`)

Implementa una sola actividad con diseño de borde a borde (`enableEdgeToEdge()`). Gestiona el estado de sesión de forma reactiva en memoria:

```
var sesionActiva by remember { mutableStateOf(sessionManager.haySesionActiva()) }
if (sesionActiva) {
    PerfilScreen(onCerrarSesion = { sesionActiva = false })
} else {
    AuthScreen(onLoginExitoso = { sesionActiva = true })
}
```

Cuando el usuario inicia sesión o la cierra, la variable `sesionActiva` cambia de valor y Compose recompone de inmediato la vista correspondiente sin necesidad de recargar la actividad ni usar enrutadores pesados.

B. Capa de Datos y Persistencia SQLite (`Usuario.kt`, `UsuarioDao.kt`, `AppDatabase.kt`)

- **Entidad (`Usuario.kt`):** Modela la tabla `usuarios`. Almacena `id` (autoincremental), `email`, `passwordHash`, `realHandle` (identidad real de 1er grado), `anonHandle` (identidad anónima de 2do grado) y `nfcUuid` (identificador criptográfico para la antena).
- **DAO (`UsuarioDao.kt`):** Interfaz con corrutinas (`suspend fun`) para inserciones y consultas parametrizadas seguras contra inyección SQL:
  - `registrarUsuario(usuario: Usuario): Long`: Inserta el registro abortando duplicados.
  - `obtenerPorEmail(email: String): Usuario?`: Valida credenciales de inicio de sesión.
  - `obtenerPorHandle(realHandle: String): Usuario?`: Comprueba la disponibilidad del nombre de usuario.
- **Base de Datos Singleton (`AppDatabase.kt`):** Única instancia protegida con `@Volatile` y `synchronized` para asegurar transacciones concurrentes libres de bloqueos.

C. Capa de Sesión y Criptografía de Hardware (`SessionManager.kt`)

- **RF-04 (Identificador Criptográfico NFC de Fondo):** La función `obtenerOGenerarDeviceNfcUuid()` revisa si el dispositivo ya cuenta con un identificador guardado en `SharedPreferences`. De no ser así, genera un hash de 128 bits mediante `java.util.UUID.randomUUID().toString()` y lo almacena de manera definitiva. Esto se ejecuta de forma totalmente transparente para el usuario sin requerir controles en pantalla.
- **RF-03 (Generador de Identidad Anónima Inalterable):** La función `generarIdentidadAnonima()` crea combinaciones pseudoaleatorias del tipo `@Anonimo_XX` (números y letras) que servirán como seudónimo inalterable para las vistas de 2do grado.

D. Pantalla de Autenticación y Registro (`AuthScreen.kt`)

Integra los controles interactivos con prevención preventiva de errores (UX):

- **RF-01 (Conmutador de Visibilidad de Contraseña):** Mediante una variable `passwordVisible`, el campo alterna entre `PasswordVisualTransformation()` (puntos ocultos) y `VisualTransformation.None` (texto plano). El botón `IconButton` en el `trailingIcon` permite al usuario verificar lo escrito y prevenir equivocaciones en la contraseña.
- **RF-02 (Handle Real con Prefijo '@' Fijo):** Implementado con `prefix = { Text("@") }` dentro de `OutlinedTextField`. El evento `onValueChange` filtra activamente cualquier espacio o carácter no alfanumérico en tiempo real mediante `it.filter { it.isLetterOrDigit() || it == '_' }`.
- **RF-03 (Presentación de Identidad Inalterable):** El seudónimo anónimo autogenerado se presenta dentro de un `Surface` estático con la etiqueta **"FIJO"**. Al no disponer de un campo de texto editable, se impide que el usuario altere su identidad anónima.
- **Validaciones Asíncronas:** Al pulsar "REGISTRAR IDENTIDAD", una corrutina (`scope.launch`) verifica en Room la disponibilidad de correo y handle antes de registrar.

E. Pantalla de Perfil de Identidad (`PerfilScreen.kt`)

Despliega el panel de identidades del usuario tras ingresar al sistema:

- **Tarjeta de 1er Grado (Morado Mate):** Muestra el `realHandle` con el color sobrio característico de conexiones confirmadas por NFC.

- **Tarjeta de 2do Grado (Gris Violeta Mate):** Muestra el seudónimo `anonHandle` informando cómo se verá el usuario ante contactos no verificados.
- **Tarjeta de Hardware Criptográfico:** Muestra el `nfcUuid` vinculado que se empleará con la antena en la Entrega Final.
- **Cierre de Sesión:** Limpia la memoria activa de `SharedPreferences` y redirige instantáneamente a la pantalla de acceso.

3. ESPECIFICACIÓN DEL TEMA VISUAL (MORADOS Y GRISES MATE)

Para satisfacer el requerimiento de una estética sobria, elegante y **no luminosa**, se ajustó la paleta cromática en `Color.kt` y `Theme.kt`:

- ■ **DarkBackground (#141318):** Fondo gris grafito mate profundo que elimina reflejos molestos y reduce la fatiga visual.
- ■ **MutedPurple (#7E639B):** Morado sobrio / mate para la identidad real de 1er grado (NFC) y botones de acción.
- ■ **MutedGray (#6E687A):** Gris violeta atenuado para el contenedor y seudónimos anónimos de 2do grado.
- ■ **MutedAccent (#9E98A8):** Gris perla apagado para los identificadores de hardware y antena NFC.
- ■ **TextPrimary (#E4E1E9):** Gris claro mate de alta legibilidad sin deslumbramiento.

4. RESUMEN DE REQUERIMIENTOS CUMPLIDOS

ID	Requerimiento Funcional	Control MD3 / Código	Resultado en Entrega 1
RF-01	Autenticación (Email / Password)	`OutlinedTextField` + `TrailingIcon`	Inicio y registro con conmutador de visibilidad funcional.
RF-02	Handle Real (@handle)	`OutlinedTextField` con `prefixText`	Formato forzado con prefijo '@' y filtrado alfanumérico.
RF-03	Identidad Anónima inalterable	`Surface / Text` estático	Generación automática (ej. @Anonimo_XX) de solo lectura.
RF-04	Identificador Criptográfico NFC	Proceso de fondo (`UUID`)	Asignación automática y persistencia en Room y SharedPreferences.